



PROGRAMAÇÃO WEB I

CSS

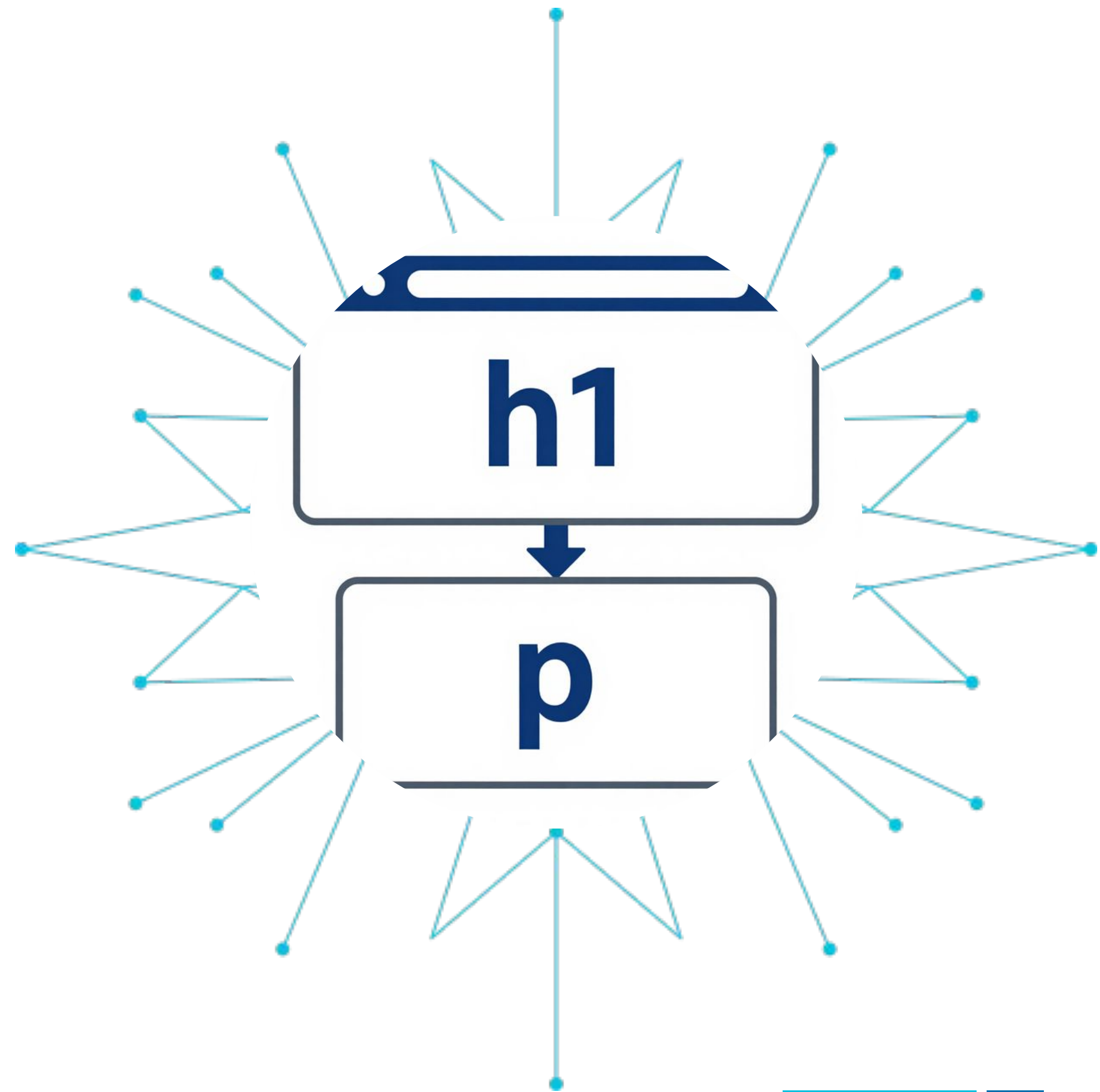
AULA INTRODUTÓRIA

Fluxo,
Dimensionamento,
Overflow e
Posicionamento



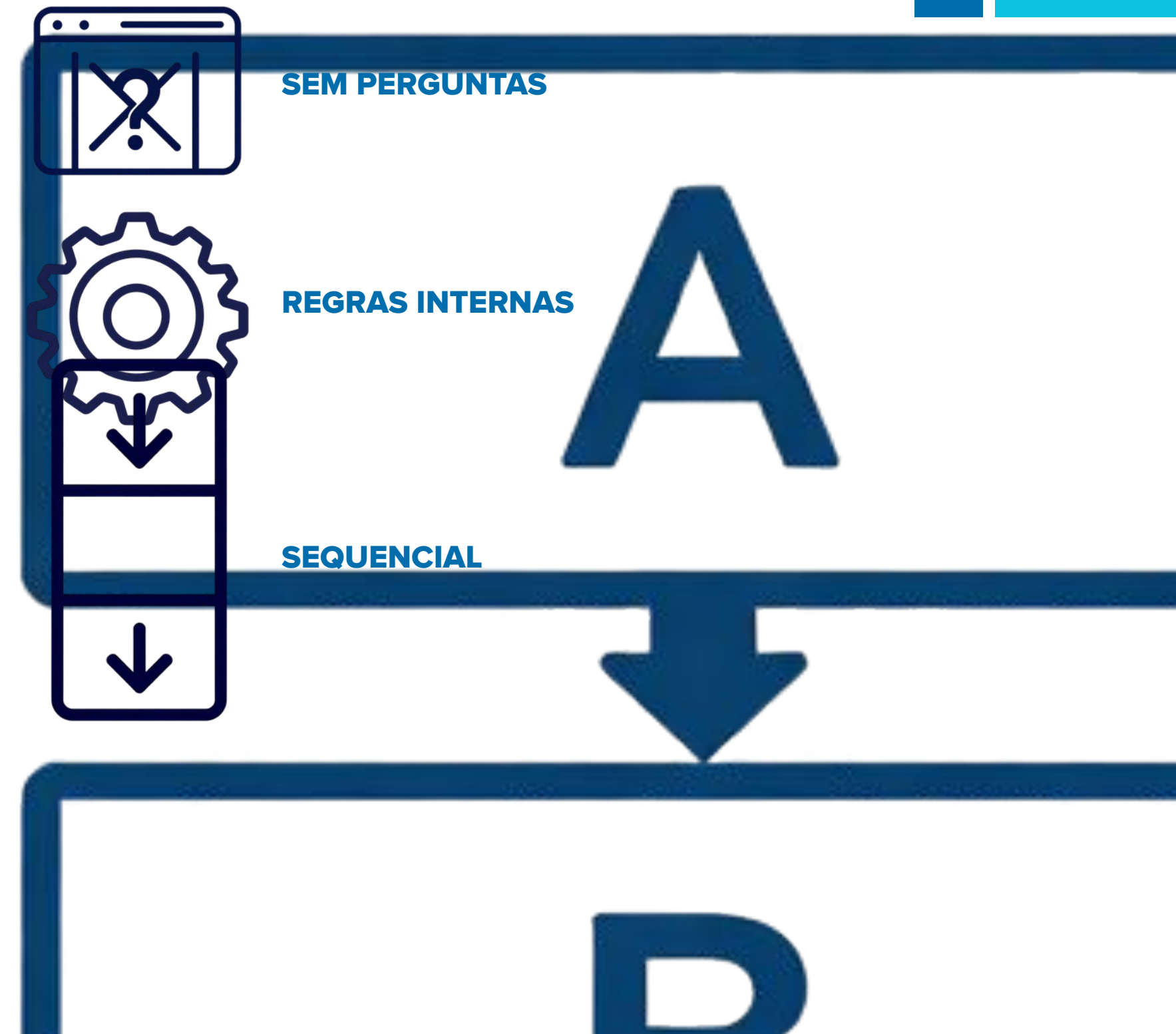
COMO O NAVEGADOR ORGANIZA OS ELEMENTOS?

Quando escrevemos HTML, como o navegador decide onde e como mostrar cada elemento? O navegador lê o código de cima para baixo e organiza os elementos no fluxo normal — um abaixo do outro, na ordem em que aparecem. Exemplo simples:



O QUE É FLUXO NORMAL?

Fluxo normal é a maneira padrão que o navegador organiza os elementos na página, um abaixo do outro, sem precisar de instruções extras de CSS.



ELEMENTOS BLOCK

Elementos block ocupam toda a largura disponível e sempre começam em uma nova linha.

Exemplos: , ,

"Eles criam uma nova linha para o próximo elemento."



ELEMENTOS INLINE

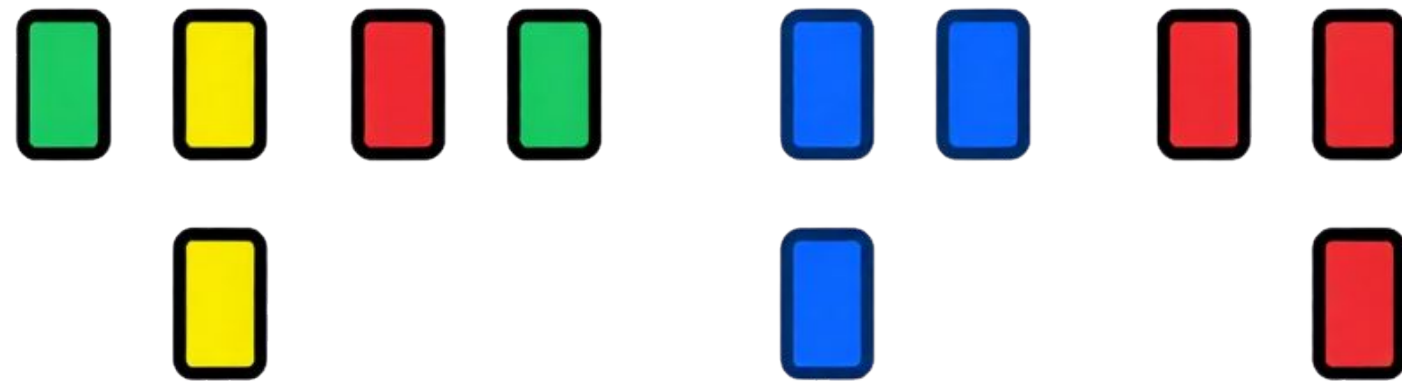
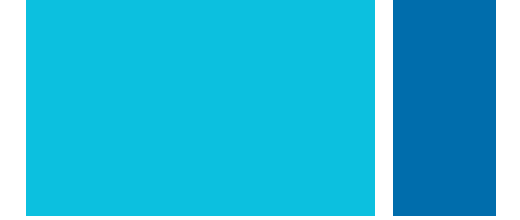
Elementos inline ficam lado a lado, dentro da mesma linha.

- Exemplos: , ,
- Não criam nova linha automaticamente
- Ocupam apenas o espaço do seu conteúdo

Ex.: "Meu nome é Gilmar e estudo CSS."

HTML as intmIL sion).

strong



BLOCK × INLINE

Compare os dois tipos de elementos e veja como se comportam na página.

Frase para memorizar:

"Block = bloco, Inline = dentro da linha"



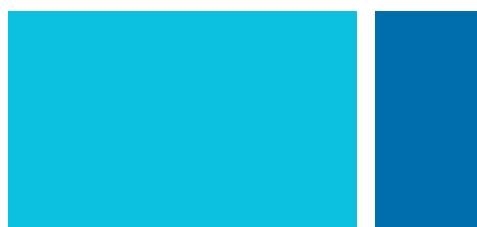
**NOVA LINHA A CADA
ELEMENTO**



**OCUPA TODA A
LARGURA DISPONÍVEL**



**PERMITE DEFINIR
LARGURA E ALTURA**



INLINE-BLOCK: O MELHOR DOS DOIS MUNDOS

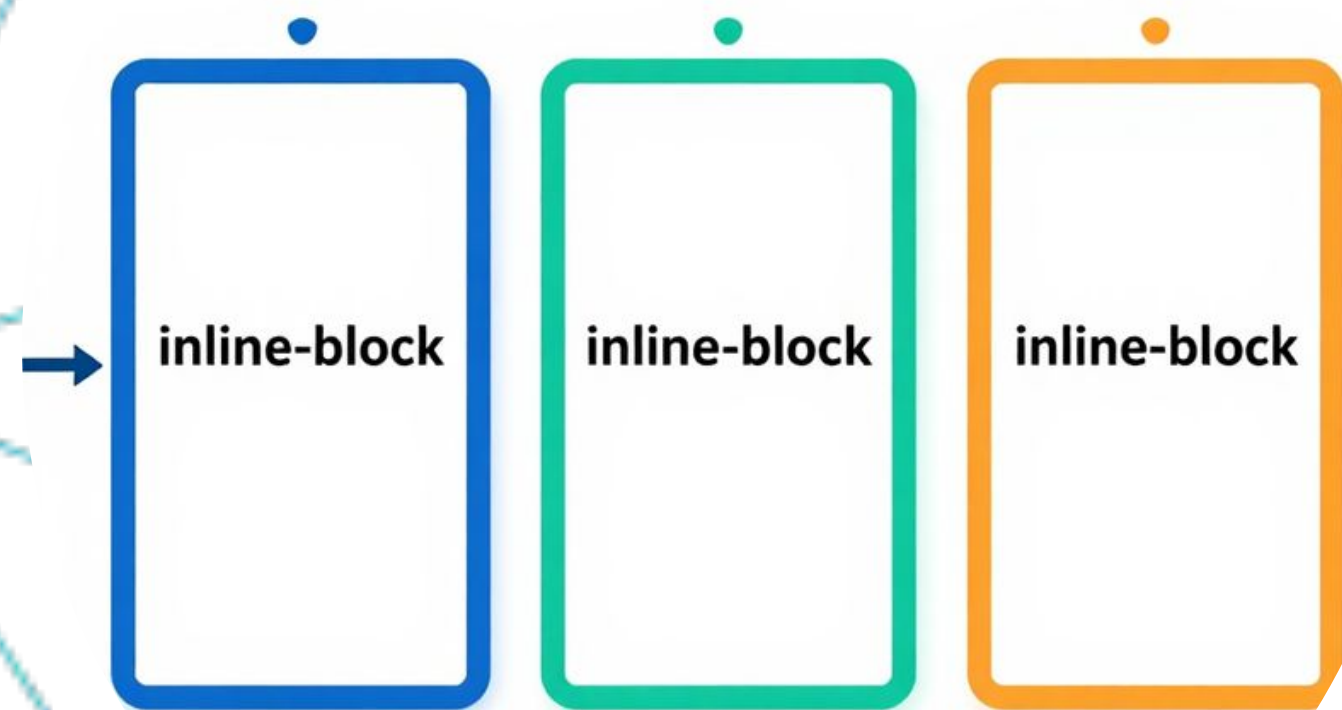
Fica lado a lado como inline, mas permite controlar largura e altura como bloco.

`display: inline-block;`

`width: 120px;`

`height: 80px;`

💡 Se não couberem na linha, quebram automaticamente para a próxima.



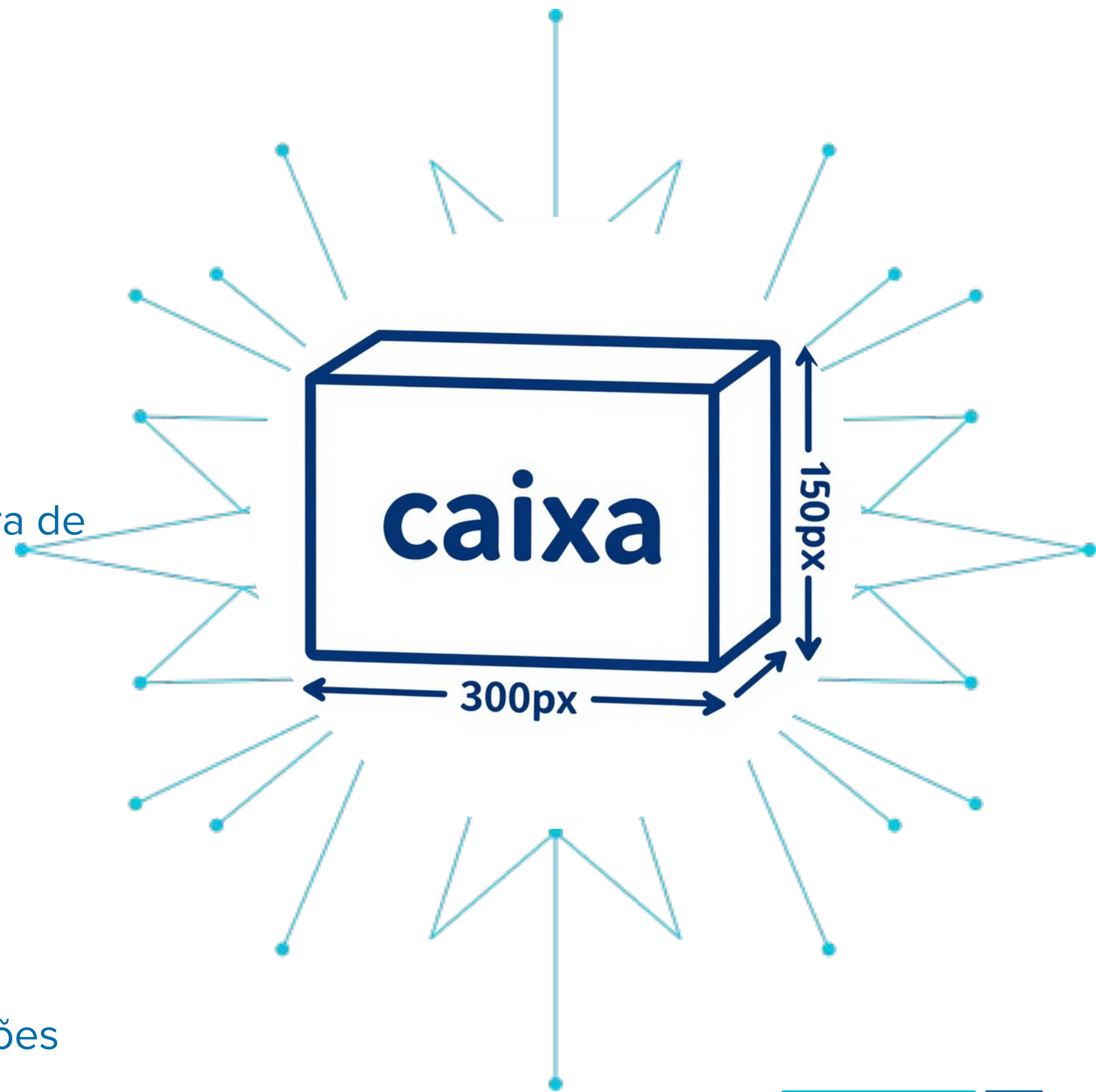
DIMENSIONAMENTO:

WIDTH E HEIGHT

width define a largura e height define a altura de um elemento.

```
.caixa {  
width: 300px;  
height: 150px;  
border: 2px solid;  
}
```

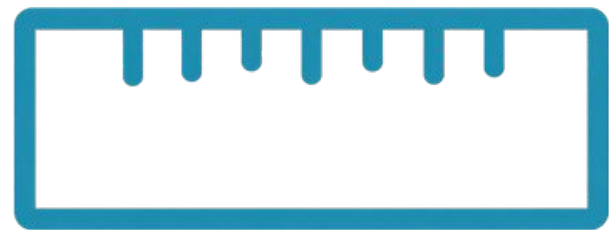
Use setas ou rótulos para indicar as dimensões visualmente.



UNIDADES DE MEDIDA EM CSS (PARTE 1)

Como definimos tamanhos em CSS? Existem unidades fixas e relativas. px é fixo e não muda. % é relativo ao elemento pai. vw e vh são relativos à janela do navegador.

Exemplo: `.caixa { width: 50%; height: 20vh; font-size: 16px; }`



PX



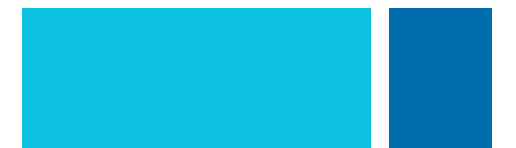
%



VW



VH



UNIDADES EM E REM

Unidades relativas à fonte controlam tamanho de forma flexível.

em: relativo à font-size do elemento pai.

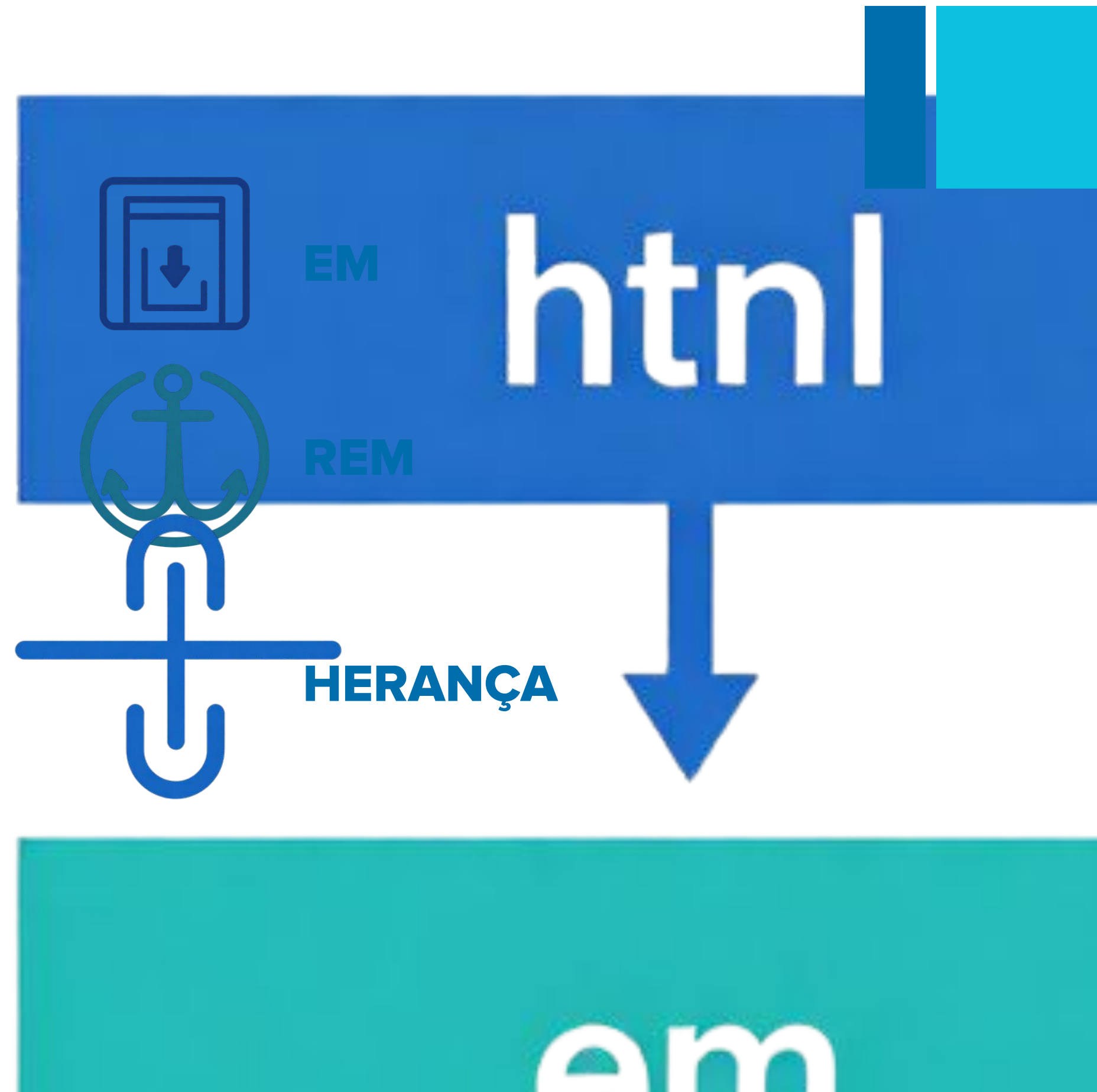
rem: relativo à font-size do elemento raiz (html).

Exemplo:

```
.caixa { font-size: 20px; padding: 2em; }
```

→ padding = 40px

Dica: prefira rem para consistência global.



min-width: 300px

max-width: 800px



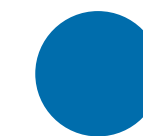
caixa



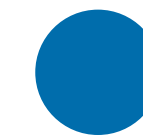
MIN-WIDTH E MAX-WIDTH

Podemos limitar o tamanho dos elementos para garantir responsividade. Assim, o layout funciona bem em telas pequenas e grandes.

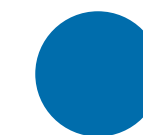
```
.caixa {  
  min-width: 300px;  
  max-width: 800px;  
  width: 100%;  
}
```



MIN-WIDTH: TAMANHO MÍNIMO



MAX-WIDTH: TAMANHO MÁXIMO



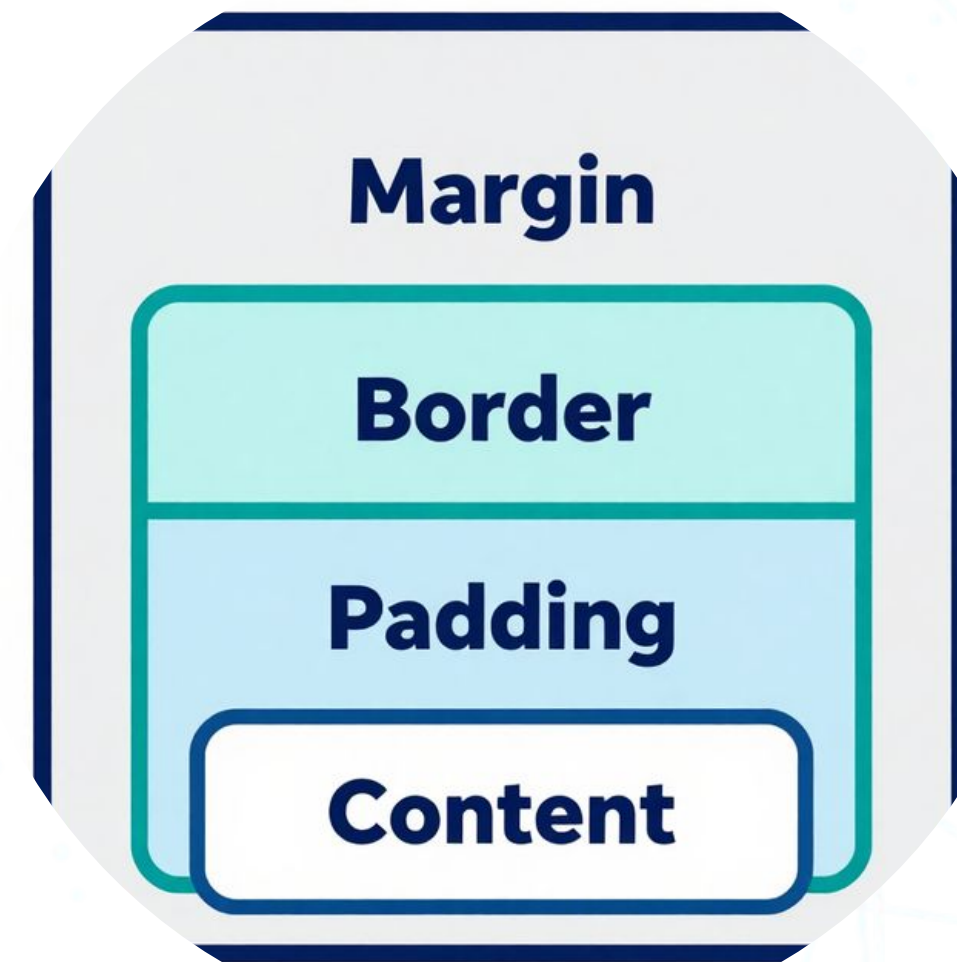
WIDTH: 100% SE ADAPTA AO PAI



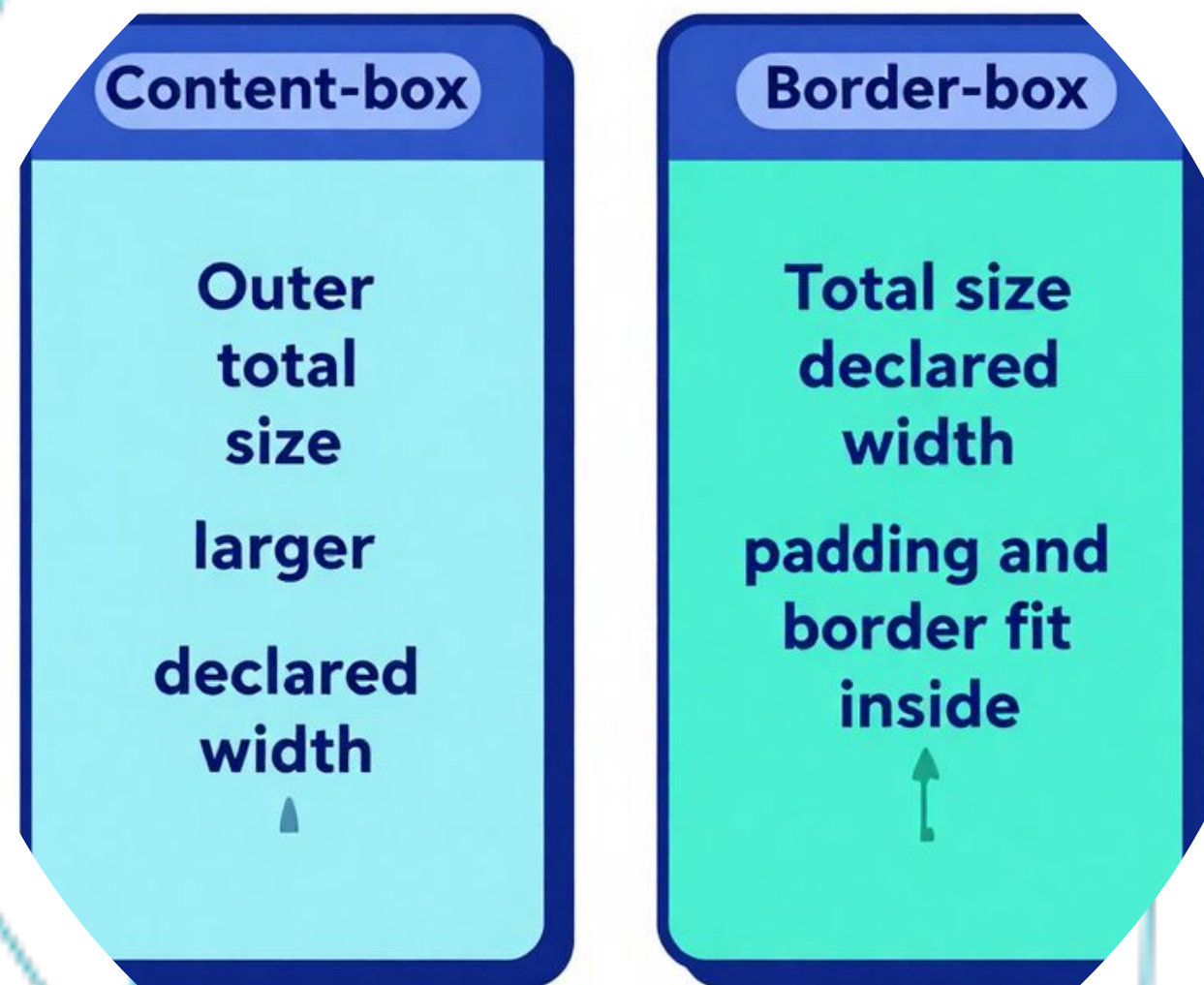
BOX MODEL – ENTENDENDO A CAIXA CSS

Todo elemento HTML é uma caixa com 4 camadas:

- Content: o conteúdo interno
- Padding: espaço entre conteúdo e borda
- Border: borda visível ao redor
- Margin: espaço externo entre



BOX-SIZING: BORDER-BOX VS CONTENT-BOX



box-sizing: content-box — width não inclui padding nem border (padrão).

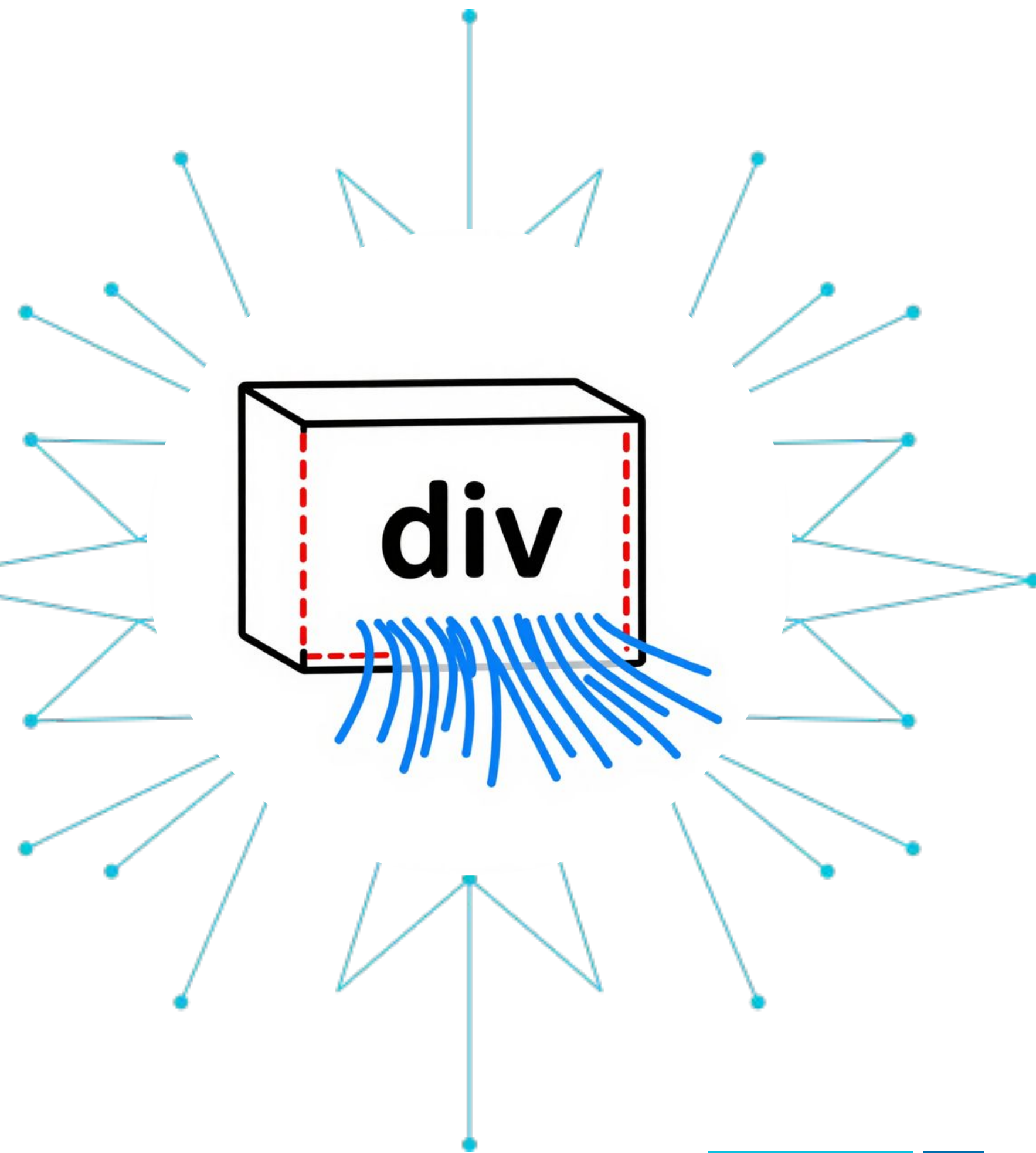
box-sizing: border-box — width inclui padding e border. Mais fácil calcular o tamanho total!

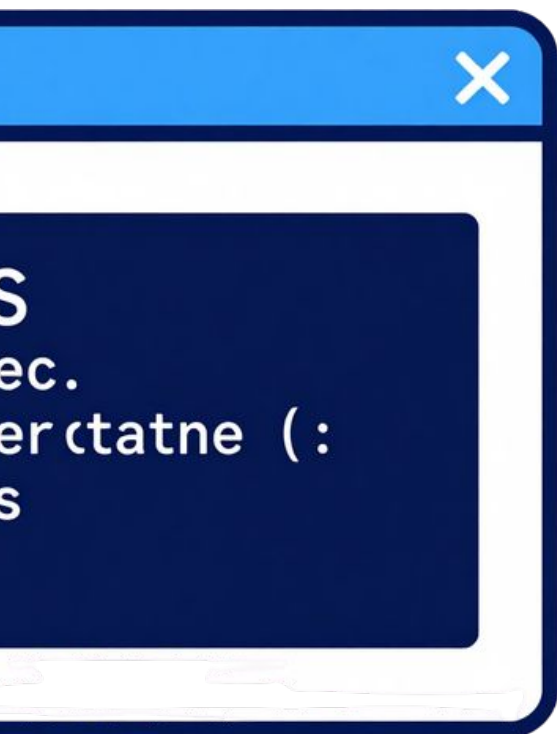
Dica: Use border-box para evitar surpresas no layout.

OVERFLOW: QUANDO O CONTEÚDO NÃO CABA

O que acontece quando o conteúdo é maior que a caixa? Por padrão, o texto ultrapassa o limite da div.

Exemplo: uma div com height: 60px contendo um parágrafo longo — o texto "vaza" para fora. Isso é overflow!





OVERFLOW: OPÇÕES DE COMPORTAMENTO

O que acontece quando o conteúdo é maior que a caixa? A propriedade overflow controla esse comportamento. Escolha o valor conforme a necessidade do layout e a experiência do usuário desejada.



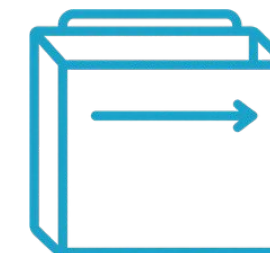
VISIBLE



HIDDEN



SCROLL



AUTO



POSITION: STATIC

Valor padrão de todos os elementos.

- Elementos seguem o fluxo normal da página
- Não aceita top, left, right ou bottom
- Comportamento natural do navegador

Exemplo CSS:

```
div { position: static; }
```

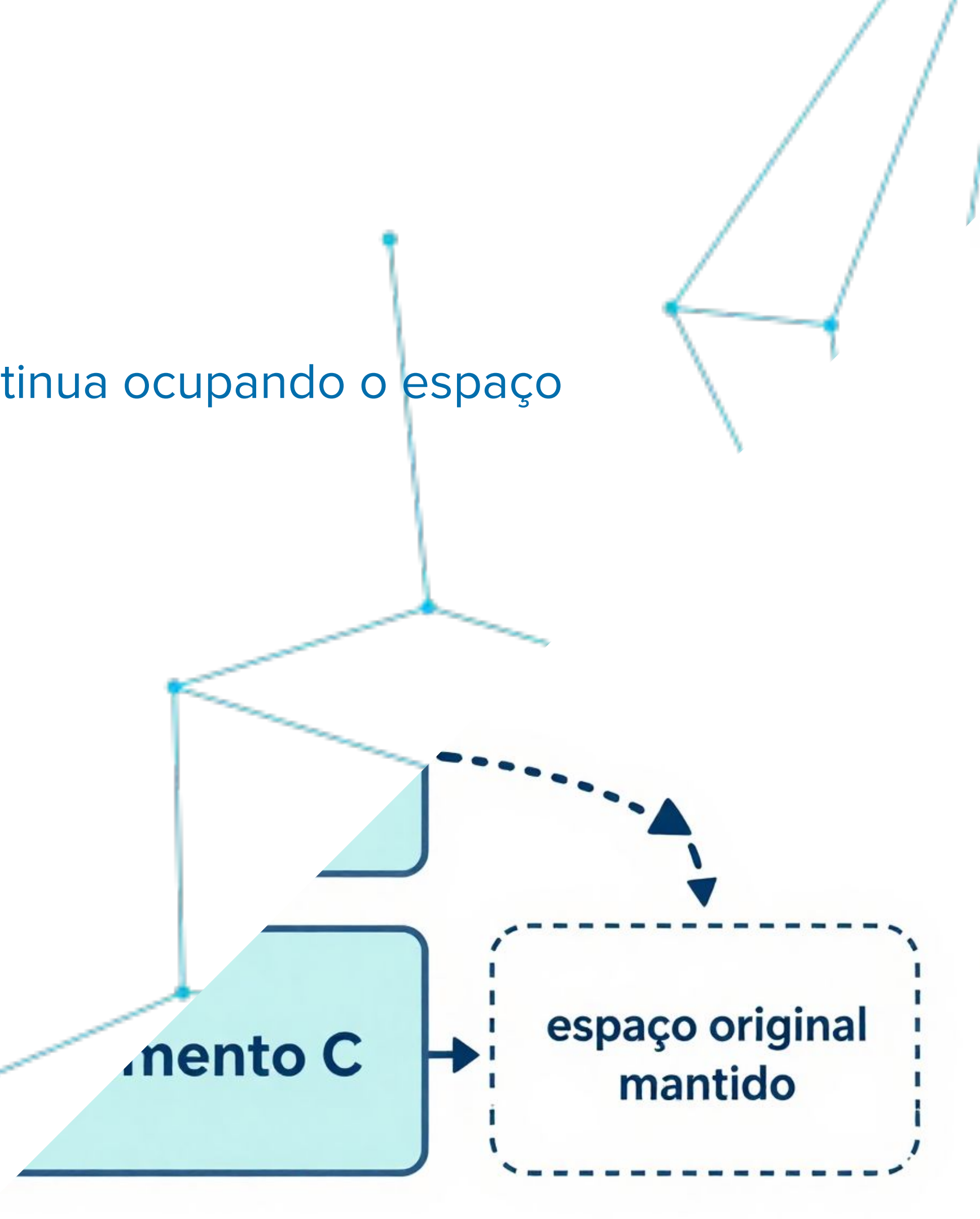


POSITION: RELATIVE

O elemento se desloca visualmente, mas continua ocupando o espaço original no fluxo.

```
.caixa {  
  position: relative;  
  top: 20px;  
  left: 10px;  
}
```

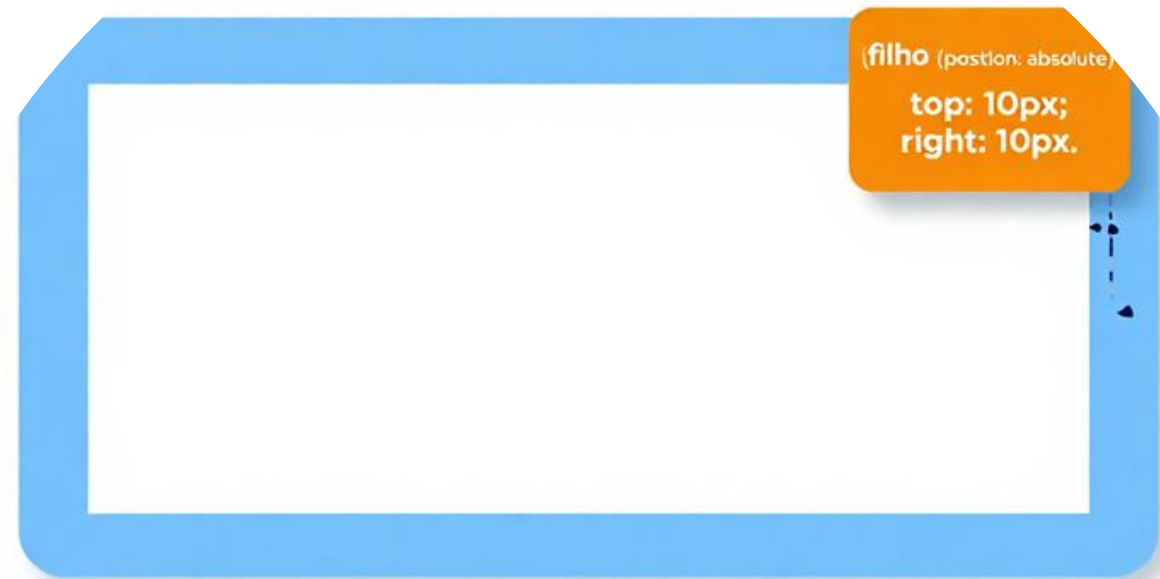
- O espaço original é preservado
- Deslocamento usa: top, left, right, bottom
- Não sai do fluxo normal



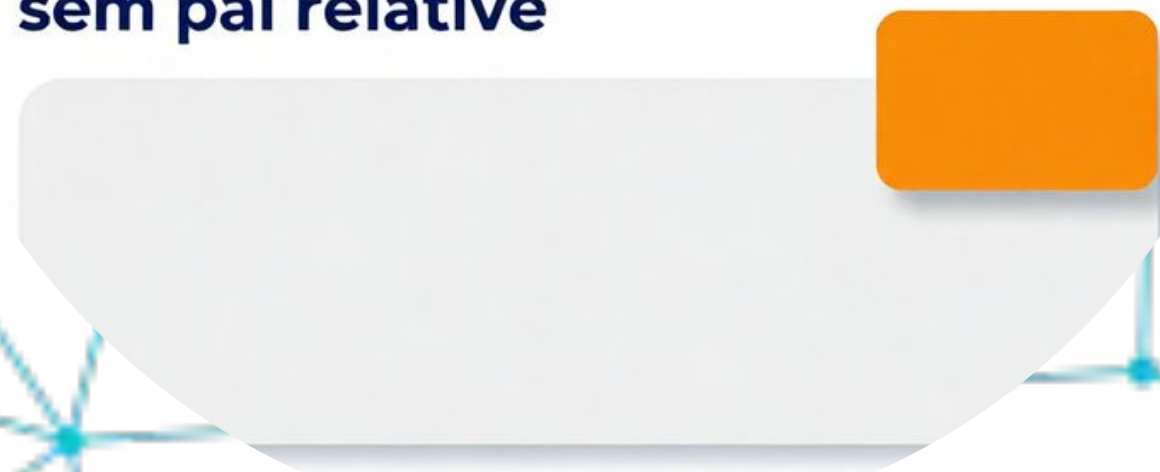
POSITION: ABSOLUTE



position: relative)



sem pai relative



O elemento sai do fluxo normal e é posicionado em relação ao ancestral com position: relative.

```
.pai { position: relative; }
```

```
.filho { position: absolute;  
top: 10px; right: 10px; }
```

Sem pai relative → posiciona em relação à janela.

POSITION: FIXED

Elementos com `position: fixed` saem do fluxo e ficam fixos na janela do navegador — não se movem ao rolar a página.

Exemplo CSS:

```
.botao-topo {  
  position: fixed;  
  bottom: 20px;  
  right: 20px;  
}
```

💡 Use para: menus fixos, botões flutuantes e barras de navegação persistentes.

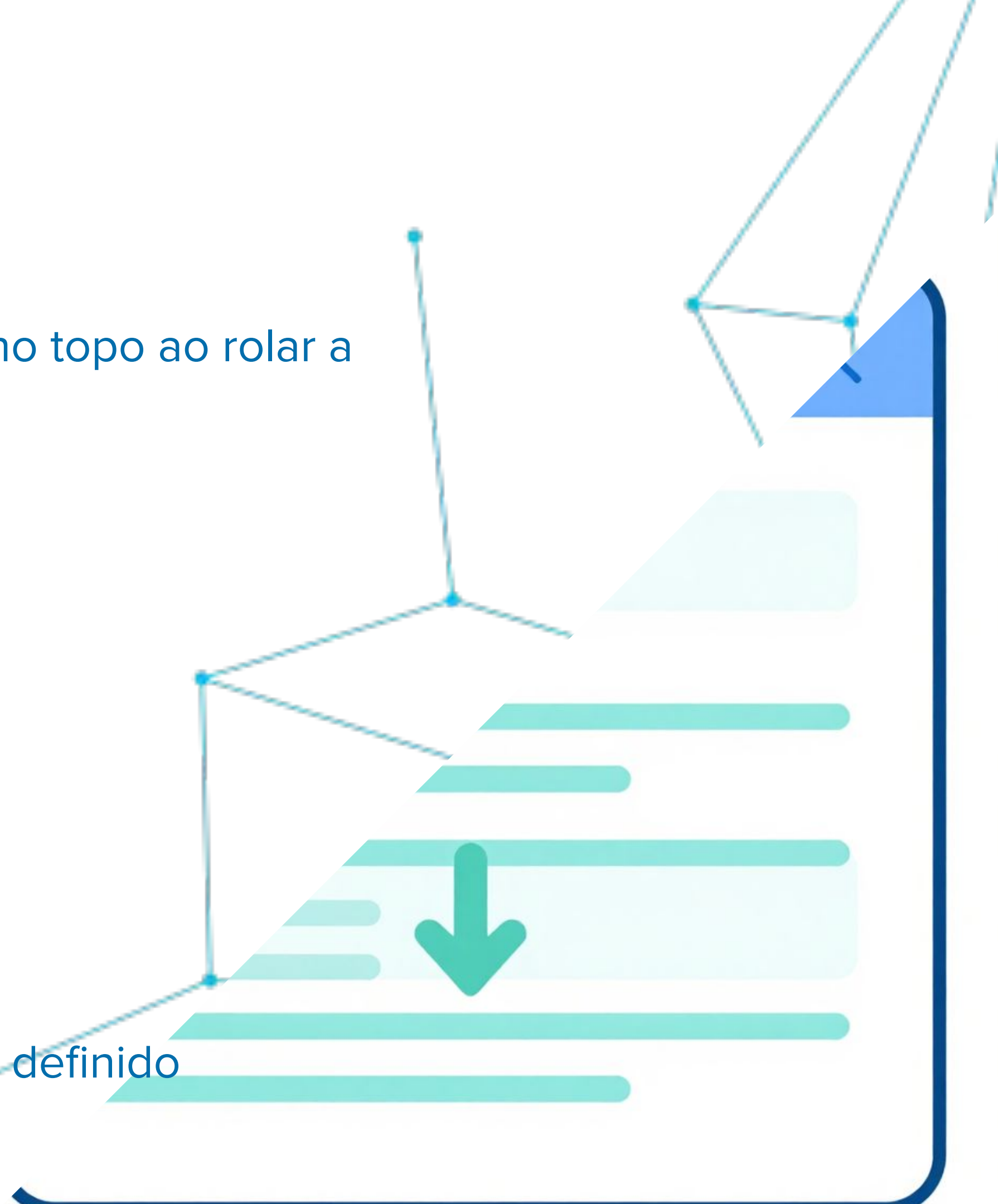
POSITION: STICKY

O elemento começa no fluxo normal e "gruda" no topo ao rolar a página.

Exemplo CSS:

```
nav {  
  position: sticky;  
  top: 0;  
  background: white;  
  border-bottom: 1px solid #ccc;  
}
```

- Comporta-se como relative até atingir o ponto definido
- Após rolar, age como fixed



Z-INDEX

SOBREPOSIÇÃO

z-index determina quem fica na frente quando elementos se sobrepõem.

```
.caixa1 { position: absolute; z-index: 1; }  
.caixa2 { position: absolute; z-index: 2;  
}
```

⚠ Só funciona em elementos com position diferente de static.

z-index: 1

z-index: 2

Re
Cem

Pop com siib
coraomenint de

Pomercenbient. can
aear tecntiiohett.



EXEMPLO PRÁTICO

Cartão de Produto com etiqueta

PROMOÇÃO em position: absolute.

```
.card { position: relative; width: 280px;  
padding: 20px; border: 1px solid #ccc; }
```

```
.etiqueta { position: absolute; top: 10px; right:  
10px; background: #e74c3c; color: #fff;  
padding: 4px 8px; }
```

A etiqueta fica no canto superior direito do
card, independente do fluxo normal.

BOTÃO FIXO

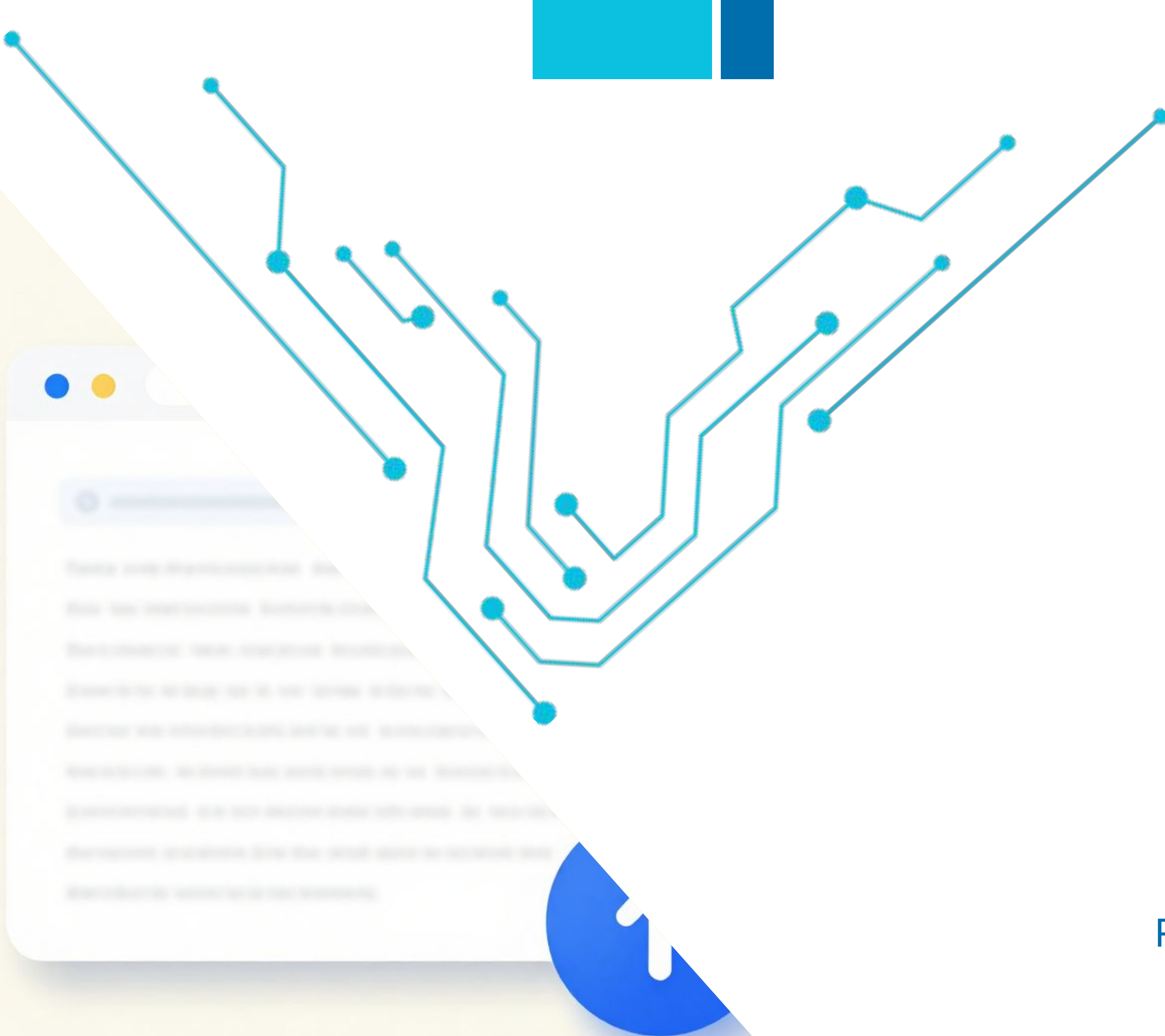
HTML: ↑ Topo

CSS:

```
#topo {  
  position: fixed;  
  bottom: 20px;  
  right: 20px;  
  padding: 12px 16px;  
}
```

Permane visível ao rolar a página.

Use position: fixed para fixar na



EXEMPLO PRÁTICO

Menu Sticky: gruda no topo ao rolar

HTML: com links de navegação

CSS:

```
nav {
```

```
  position: sticky;
```

```
  top: 0;
```

```
  background: white;
```

```
  border-bottom: 2px solid #ccc;
```

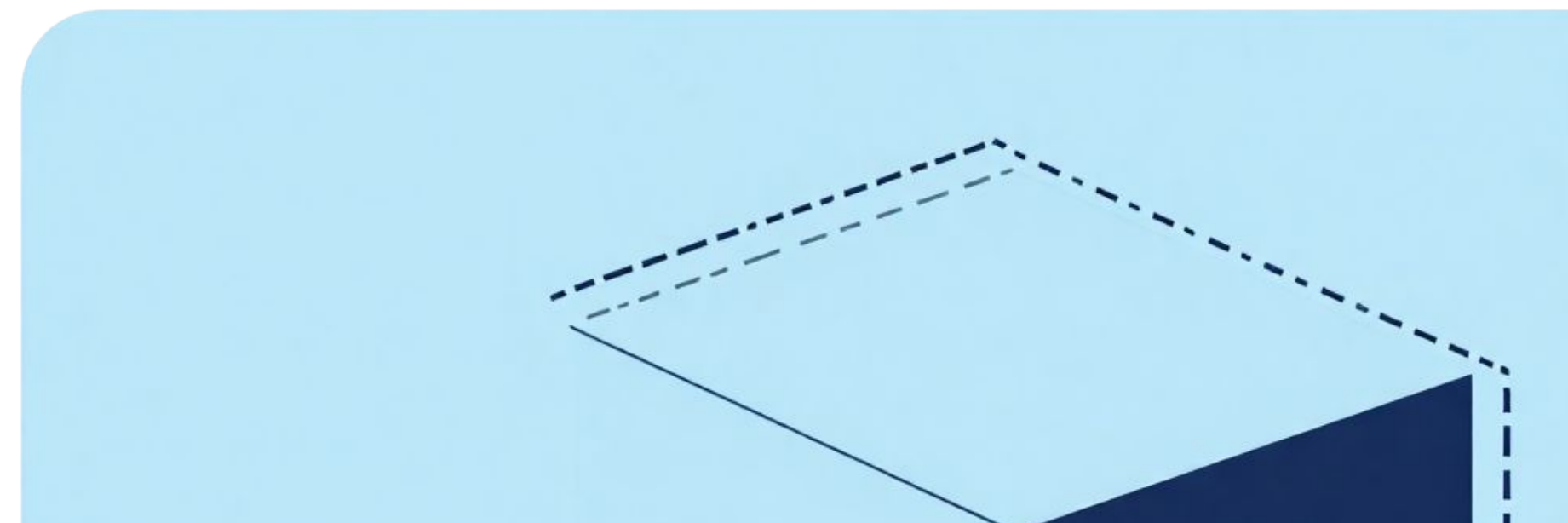
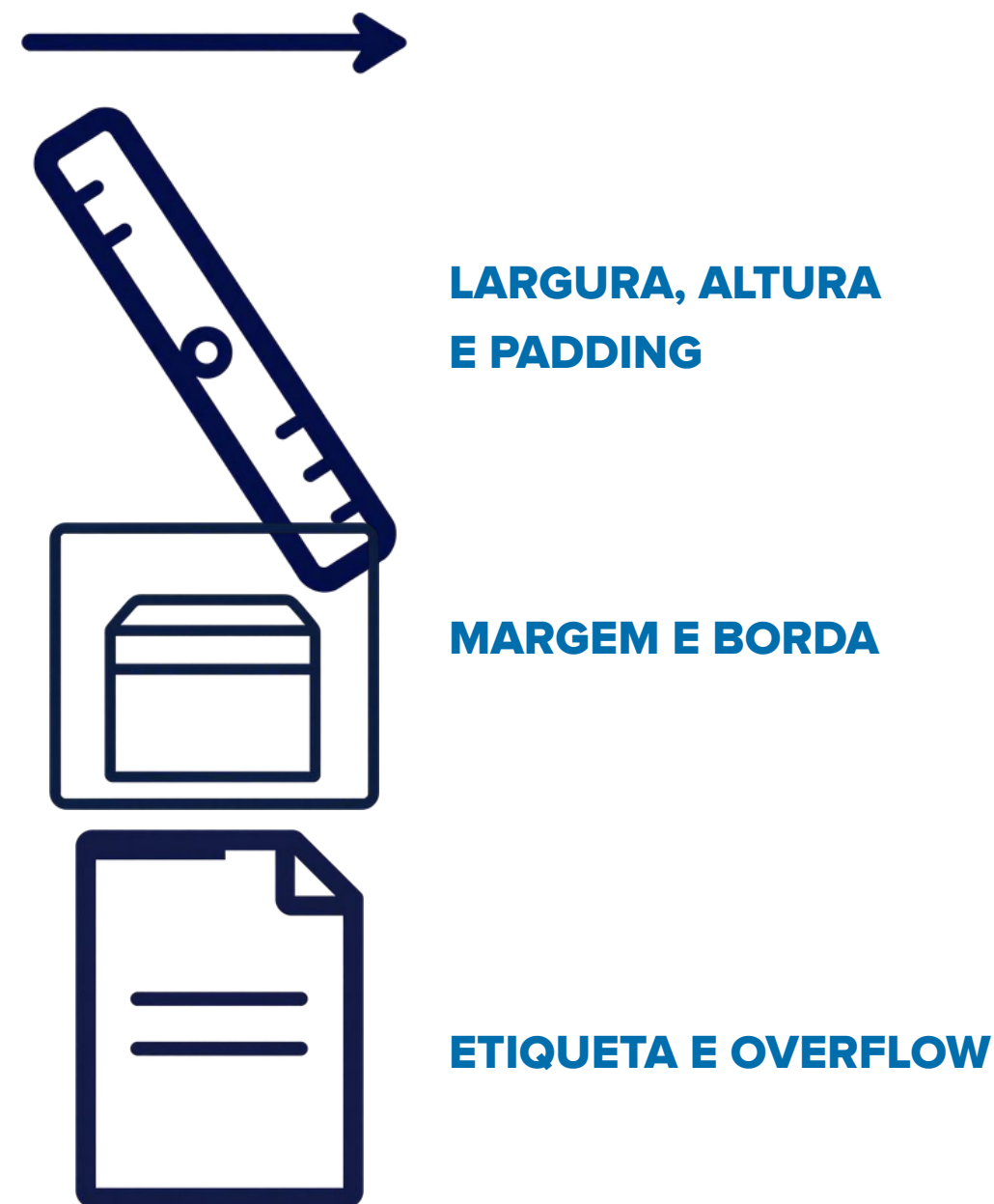
```
}
```

O menu começa no fluxo normal e fica fixo
no topo ao rolar a página



DESAFIO PRÁTICO 1

Modifique o cartão criado em aula. Altere as propriedades abaixo e observe como o layout muda no navegador. Experimente valores diferentes e veja o resultado!



DESAFIO

PRÁTICO 2

Coloque em prática o que você aprendeu sobre position: fixed!

Siga as etapas abaixo e observe o comportamento do botão ao rolar a página.



**CRIE O BOTÃO
FIXO**



**ADICIONE CONTEÚDO
LONGO**



OBSERVE O COMPORTAMENTO



DESAFIO 3

MENU STICKY

Aplique position: sticky para criar um menu que "gruda" no topo ao rolar a página. Siga os passos ao lado e observe o comportamento no navegador.



CRIE O MENU



**ADICIONE
CONTEÚDO**



OBSERVE O STICKY





FLUXO E DISPLAY

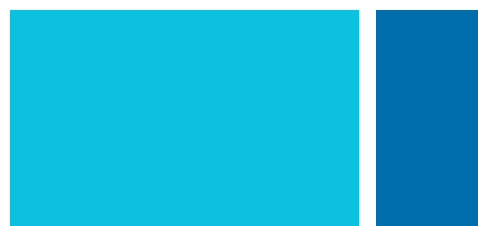
BOX MODEL

POSICIONAMENTO

PERGUNTAS DE VERIFICAÇÃO

Responda mentalmente cada pergunta para checar sua compreensão:

- O que é fluxo normal?
- Diferença entre block e inline?
- O que definem width e height?
- Para que servem padding e margin?
- O que faz overflow: hidden?
- Diferença entre position relative e absolute?
- Qual position fixa o elemento na janela do



RESUMO VISUAL – MAPA MENTAL CSS BÁSICO

Fluxo: static (padrão)

Elementos: block | inline | inline-block

Tamanho: width, height, min/max-width

Box Model: content → padding → border → margin

Overflow: visible | hidden | scroll | auto

Position: static | relative | absolute | fixed | sticky

Z-index: controla sobreposição de





CONTATO

CSS controla tamanho, espaço e posição.

Próximos passos: Flexbox, Grid e
Responsividade.

Pratique os desafios e explore!

@profgilmarfreire

